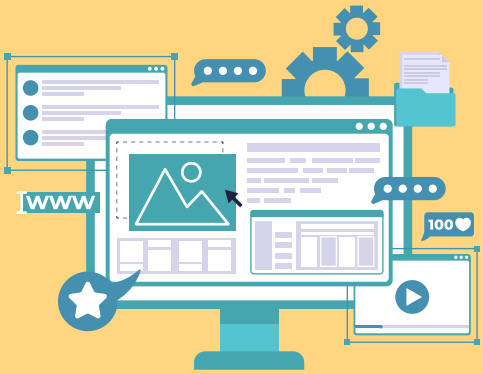
Myat Noe Angel OSSD Batch 4
ICS4U Rich Summative Task

CLASSES, OBJECTS AND METHODS



What Is a Class and an Object?

- Class: A blueprint/template for objects.
- Object: An instance of a class.

Constructors

- Special method called when an object is created.
- No return type.
- You can have multiple constructors (overloading).

this Keyword

- Refers to the current object.
- Resolves name conflicts.

Access Modifiers

Modifier	Meaning
public	Accessible from anywhere
private	Accessible only inside the class
protected	Accessible in same package or subclass

Statics vs. Instance Variables

- Instance variables belong to the object.
- Static variables belong to the class (shared).

Methods

- Void Method: Does not return a value.
- Return Method: Returns a value.

Encapsulation (Getters/Setters)

- Keep fields private
- Access them through public methods

Inheritance

- Subclass inherits fields/methods from superclass
- Use extends keyword

Method Overloading vs Overriding

- Overloading: Same name, different parameters
- Overriding: Subclass provides a new implementation of a superclass method

Recursion

- A method calling itself.
- Must have a base case to avoid infinite loop.

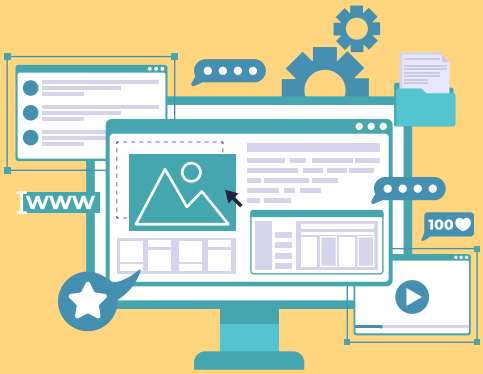
toString() Method

- Customise how your object prints

Java

Myat Noe Angel OSSD Batch 4
ICS4U Rich Summative Task

BOOLEAN EXPRESSIONS AND IF STATEMENTS



What Are Boolean Expressions?

- Boolean: A data type with only two possible values: true or false.
- Boolean expression: An expression that evaluates to either true or false.

Relational Operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Logical Operators

Operator	Name	Description
&&	AND	Returns true if both statements are true
||	OR	Returns true if one of the statements is true
!	NOT	Reverse the result

De Morgan's Laws

- !(a && b) is equivalent to !a || !b
- !(a || b) is equivalent to !a && !b

Conditional Statements

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false

Common Errors

- Using = instead of ==

Solution: if (x == 5) not if (x = 5)

- Missing braces {} for multiple lines

Solution: Always use {} for clarity

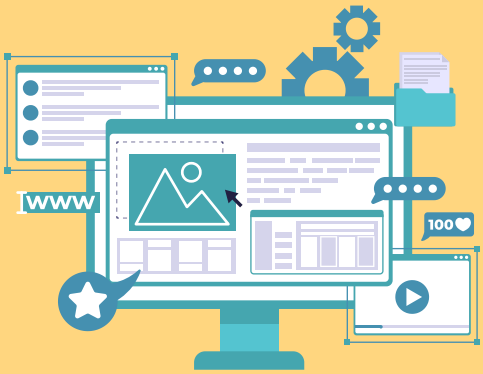
- Forgetting ! affects entire expression

Solution: Use parentheses for grouping

Java

Myat Noe Angel OSSD Batch 4
ICS4U Rich Summative Task

ITERATION



Iteration (looping) allows executing a block of code multiple times until a condition is met.

for Loop

- Used when the number of iterations is known.

```
Syntax:
for (initialisation;
condition; update) {
  // code block
}
```

while Loop

- Used when the number of iterations is unknown. It checks the condition before running.

```
Syntax:
while (condition) {
  // code block
}
```

do-while Loop

- Executes the block at least once, then checks the condition.

```
Syntax:
do {
  // code block
} while (condition);
```

Nested Loops

- Loops inside loops are called nested loops.
- Useful for 2D data structures or repeated tasks.

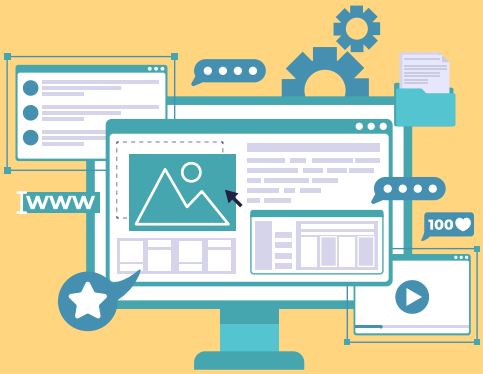
Loop Control Keywords

- break: Exit the loop immediately
- continue: Skip the current iteration and move to the next one

Java

Myat Noe Angel OSSD Batch 4
ICS4U Rich Summative Task

ARRAYS AND ARRAYLISTS



Arrays

- Fixed-size container for elements of the same data type.
- Index starts at 0.

Declaring and Initialising:

```
int[] numbers = new int[5]; // default values
int[] scores = {90, 85, 70, 88}; // directly initialized
```

Traversal with a for loop:

```
for (int i = 0; i < scores.length; i++) {
    System.out.println("Score: " + scores[i]);
}
```

Traversal with a for-each loop:

```
for (int score : scores) {
    System.out.println("Score: " + score);
}
```

ArrayLists

- Resizable array from the Java Collections Framework.
- Only stores objects, not primitives (use wrapper classes like Integer, Double, etc.)

Import and Declare:

```
import java.util.ArrayList;

ArrayList<String> names = new ArrayList<>();
```

Adding/Removing Elements:

```
names.add("Alice");
names.add("Bob");
names.remove("Alice"); // by value
names.remove(0); // by index
```

Getting/Setting Elements:

```
String name = names.get(0); // Get element at index 0
names.set(0, "Charlie"); // Set new value at index 0
```

Size of ArrayList:

```
int size = names.size();
```

Feature	Array	ArrayList
Size	Fixed	Dynamic
Data Type	Primitives & Objects	Objects only
Performance	Slightly faster	More flexible
Syntax Simplicity	Basic	Richer methods